

Los Sabrossos

Contents

1	STRINGS	3
1.1	Hashing	3
2	TREE ALGORITHMS	3
2.1	Binary lifting	3
2.2	Lowest common ancestor	3
3	DATA STRUCTURES	4
3.1	Disjoint set union	4
3.2	Sparse table	5
4	MATH	5
4.1	Binary exponentiation	5
4.2	Sieve of eratosthenes	6
5	GRAPH ALGORITHMS	6
5.1	Bfs	6
5.2	Dfs	6
5.3	Dijkstra	7
5.4	Kruskal mst	7

1 STRINGS

1.1 Hashing

```
1 struct Hash {
2     long long P=1777771,MOD[2],PI[2];
3     vector<long long> h[2],pi[2];
4     Hash(string& s){
5         MOD[0]=999727999;MOD[1]=1070777777;
6         PI[0]=325255434;PI[1]=10018302;
7         for(int k = 0; k < 2; k++) h[k].resize(s.size()+1),pi[k].resize(s.size()+1);
8         for(int k = 0; k < 2; k++){
9             h[k][0]=0;pi[k][0]=1;
10            long long p=1;
11            for(int i = 1; i < s.size()+1; i++){
12                h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
13                pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
14                p=(p*P)%MOD[k];
15            }
16        }
17    }
18    long long get(int s, int e){
19        long long h0=(h[0][e]-h[0][s]+MOD[0])%MOD[0];
20        h0=(1LL*h0*pi[0][s])%MOD[0];
21        long long h1=(h[1][e]-h[1][s]+MOD[1])%MOD[1];
22        h1=(1LL*h1*pi[1][s])%MOD[1];
23        return (h0<<32)|h1;
24    }
25 };
```

2 TREE ALGORITHMS

2.1 Binary lifting

```
1 const int N = 1e5;
2 const int LOG = 20;
3 int up[N][LOG];
4 vector<int> G[N];
5 //Find parents
6 void dfs(int u){
7     for(int v : G[u]){
8         up[v][0] = u;
9         for(int j = 1; j < LOG; j++){
10             up[v][j] = up[up[v][j-1]][j-1];
11         }
12         dfs(v);
13     }
14 }
15 int get_K_parent(int a, int k){
16     for(int j = LOG-1; j >= 0; j++){
17         if(k&(1<<j)){
18             a = up[a][j];
19         }
20     }
21     return a;
22 }
```

2.2 Lowest common ancestor

```

1  const int N = 1e5;
2  const int LOG = 20;
3  vector<int> G[N];
4  int depth[N];
5  int up[N][LOG];
6  // Assign levels and get parents
7  void dfs(int u){
8      for(int v : G[u]){
9          depth[v] = depth[u] + 1;
10         up[v][0] = u;
11         for(int j = 1; j < LOG; j++){
12             up[v][j] = up[up[v][j-1]][j-1];
13         }
14         dfs(v);
15     }
16 }
17 int get_lca(int a, int b){
18     if(depth[a] < depth[b]) swap(a,b);
19     int k = depth[a] - depth[b];
20     for(int j = LOG-1; j >= 0; j--){
21         if(k & (1<<j)){
22             a = up[a][j];
23         }
24     }
25     if(a == b) return a;
26     for(int j = LOG-1; j >= 0; j--){
27         if(up[a][j] != up[b][j]){
28             a = up[a][j];
29             b = up[b][j];
30         }
31     }
32     return up[a][0];
33 }

```

3 DATA STRUCTURES

3.1 Disjoint set union

```

1  const int N = 1e5;
2  struct DSU{
3      int num;
4      vector<int> parent;
5      vector<int> sizes;
6      DSU(){};
7      DSU(int n){
8          init(n);
9      }
10     void init(int n){
11         num = n;
12         parent.resize(n+1);
13         sizes.resize(n+1);
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }

```

```

23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y]+=sizes[x];
31         }
32         else{
33             parent[y] = x;
34             sizes[x]+=sizes[y];
35         }
36     }
37 }
38 };

```

3.2 Sparse table

```

1  const int N = 1e5;
2  const int LOG = 20;
3  int v[N]; //values
4  int st[N][LOG]; //Sparse Table
5  int lg[N]; // log values
6  int n;
7  void build(){
8      for(int i = 0; i < n; i++) st[i][0] = v[i];
9
10     for(int i = 1; i < LOG; i++){
11         for(int j = 0; j < n - (1<<i) + 1; j++){
12             st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
13         }
14     }
15     lg[1] = 0;
16     for(int i = 2; i <= n;i++) lg[i] = lg[i/2] + 1;
17 }
18 int query(int L, int R){
19     int k = lg[R-L+1];
20     return min(st[L][k],st[R-(1<<k)+1][k]);
21 }
22 // v = [5, 2, 4, 7, 1, 3]
23 // query(1, 4) → [2, 4, 7, 1]

```

4 MATH

4.1 Binary exponentiation

```

1  const ll mod = 998244353;
2  ll pote(ll a, ll b){
3      ll ans = 1ll;
4      while(b){
5          if(b&1) ans = (ans * a)%mod;
6          b>>=1;
7          a = (a * a)%mod;
8      }
9      return ans;
10 }

```

4.2 Sieve of eratosthenes

```
1  const int N = 1e7;
2  int sieve[N];
3  void make(){
4      for(int i = 0; i < N; i++) sieve[i] = 1;
5      sieve[0] = sieve[1] = 0;
6      for (int i = 2; i < N; i++)
7      {
8          if(sieve[i]){
9              for (int j = i*i; j < N; j+=i)
10                 {
11                     sieve[j] = 0;
12                 }
13             }
14         }
15     }
```

5 GRAPH ALGORITHMS

5.1 Bfs

```
1  const int N = 1e5;
2
3  vector<int> G[N];
4  int vis[N];
5  void bfs(int source){
6      queue<int> q;
7      q.push(source);
8      vis[source] = 0;
9      while(!q.empty()){
10         int u = q.front();
11         q.pop();
12         for(int v : G[u]){
13             if(vis[v] == -1){
14                 vis[v] = vis[u] + 1;
15                 q.push(v);
16             }
17         }
18     }
19 }
```

5.2 Dfs

```
1  const int N = 1e5;
2
3  vector<int> G[N];
4  int vis[N];
5
6  void dfs(int u){
7      vis[u] = 1;
8      for(int v : G[u]){
9          if(vis[v] == -1){
10             dfs(v);
11         }
12     }
13 }
```

5.3 Dijkstra

```
1  const int N = 1e5;
2
3  int vis[N];
4  vector<pair<int,int>> G[N];
5
6  void dijkstra(int x){
7      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
8      pq.push({0,x});
9      vis[x] = 0;
10     while(!pq.empty()){
11         auto y = pq.top();
12         pq.pop();
13         int u = y.second;
14         int cost = y.first;
15         for(auto v : G[u]){
16             int cur = cost + v.second;
17             if(cur < vis[v.first]){
18                 vis[v.first] = cur;
19                 pq.push({cur,v.first});
20             }
21         }
22     }
23 }
```

5.4 Kruskal mst

```
1  const int N = 1e5;
2  struct DSU{
3      int num;
4      vector<int> parent;
5      vector<int> sizes;
6      DSU(){};
7      DSU(int n){
8          init(n);
9      }
10     void init(int n){
11         num = n;
12         parent.resize(n+1);
13         sizes.resize(n+1);
14         for(int i = 1; i <= n ; i++){
15             parent[i] = i;
16             sizes[i] = 1;
17         }
18     }
19     int find(int a){
20         if(a == parent[a]) return a;
21         return parent[a] = find(parent[a]);
22     }
23     void join(int a, int b){
24         int x = find(a);
25         int y = find(b);
26         if(x == y) return;
27         num--;
28         if(sizes[x] < sizes[y]){
29             parent[x] = y;
30             sizes[y] += sizes[x];
31         }
32         else{
33             parent[y] = x;
```



```

34         sizes[x]+=sizes[y];
35     }
36 }
37 };
38 int n;
39 vector<pair<int,pair<int,int>>> List;
40 int Kruskal_MST(){
41     sort(List.begin(),List.end());
42     DSU dsu(n);
43     int mst = 0, t = 1;
44     for(int i = 0; i < List.size(); i++){
45         int w = List[i].first; //weight
46         int a = List[i].second.first; //at
47         int b = List[i].second.second; //to
48         if(dsu.find(a) != dsu.find(b)){
49             dsu.join(a,b);
50             mst+=w;
51             t++;
52         }
53         if(t == n) break;
54     }
55     return mst;
56 }

```